

CVA6 VS FP-State Lazy-Save Disclosure Vulnerability Report

Affected version

Issue package: ../../issues/CV-6

This package was validated on a local CVA6 checkout at revision v5.3.0-168-gcd4c8b17, confirmed at commit cd4c8b17c102, using the cv64a6_imafdch_sv39 Hypervisor configuration. No fix revision has been identified yet.

Background

The issue package states the implementation bug: in VS-mode, immediately after an FP register write, the V-visible sstatus view can report FS=Dirty while the summary dirty bit SD still reads as 0. A delayed reread later recovers to SD=1.

The security PoC in this package puts that stale summary bit into a lazy FP context-switch decision. Context A writes a marker into ft0, the VS-mode scheduler immediately reads sstatus.SD, and affected CVA6 lets the scheduler skip saving or clearing the FP register file. Context B then reads ft0 and observes context A's marker.

Vulnerability type and CWE

Vulnerability type. Stale architectural dirty-summary bit causing an incorrect lazy FP save decision and cross-context FP-state exposure.

CWE. Primary: CWE-693 Protection Mechanism Failure. Secondary: CWE-200 Exposure of Sensitive Information to an Unauthorized Actor and CWE-362 Concurrent Execution using Shared Resource with Improper Synchronization.

Core defect

When the core is in virtual mode, a VS sstatus access is translated to the VS status CSR:

```
assign conv_csr_addr = (CVA6Cfg.RVH) ? riscv::convert_vs_access_csr(
    (riscv::csr_t'(csr_addr_i)), v_q
) : csr_addr;
```

The translated CSR_VSSTATUS read returns the current vsstatus view:

```
riscv::CSR_VSSTATUS:
if (CVA6Cfg.RVH) csr_rdata = vsstatus_extended;
else read_access_exception = 1'b1;
```

An FP-writing instruction at commit marks FP state dirty. The commit-stage dirty signal is wired into csr_regfile, and the CSR logic updates vsstatus_d.fs when the core is executing in virtual mode:

```
if (CVA6Cfg.FpPresent && (dirty_fp_state_csr || dirty_fp_state_i)) begin
    mstatus_d.fs = riscv::Dirty;
    if (CVA6Cfg.RVH && v_q) begin
        vsstatus_d.fs = riscv::Dirty;
    end
end
```

But the SD summary bit is computed from the previous registered state rather than from the just-updated next state:

```
if (CVA6Cfg.RVH) begin
```

```
vsstatus_d.sd = (vsstatus_q.xs == riscv::Dirty) | (vsstatus_q.fs == riscv::Dirty);
end
```

An immediate architectural read can observe an inconsistent pair:

- FS=Dirty, because the FP write has been recognized;
- SD=0, because the summary is still derived from the old
vsstatus_q.fs=Initial value.

vsstatus.SD is derived from old q-state even when the same CSR update computes a new dirty vsstatus.FS next-state.

Security boundary and realistic scenario

The relevant boundary is FP state isolation between contexts managed by virtualized supervisor software. Lazy FP save/restore code commonly uses the status dirty bits to avoid saving extended state when a context has not touched that state.

In the vulnerable pattern:

1. Context A executes in VS-mode and writes a secret or sensitive intermediate value into an FP register.
2. The VS-mode context manager immediately checks the V-visible sstatus summary to decide whether extended FP state needs saving.
3. Affected CVA6 reports FS=Dirty but stale SD=0.
4. The context manager treats SD=0 as "no extended state is dirty" and skips save/clear.
5. Context B is scheduled on the same hardware FP register file and can observe context A's FP value.

This is a confidentiality failure for FP register state, with a closely related integrity failure if the skipped save allows a later context to corrupt state that should have been preserved.

Security PoC

Assumptions

The PoC uses a VS-mode lazy-save scheduler that checks sstatus.SD before saving or clearing FP state. Context A and context B are mutually distrusting contexts sharing the same virtualized supervisor.

PoC setup

M-mode enables the H and F paths, initializes the VS-visible status to FS=Initial and SD=0, and enters VS-mode.

Context A first verifies that baseline and then writes a recognizable marker into ft0:

```
li    t0, A_SECRET_WORD
context_a_fp_write_site:
    fmv.w.x ft0, t0
```

The scheduler immediately reads the V-visible status view:

```

lazy_save_scheduler:
    csrr s2, sstatus
    ...
    bnez t1, pass_scheduler_saw_dirty_summary
    ...
    j      vs_context_b

```

If SD=1, the PoC exits 0, representing the compliant path where the scheduler knows FP state is dirty and would save or clear it before running the next context.

If SD=0, the PoC takes the lazy-save skip path and runs context B without saving or clearing the FP register file:

```

vs_context_b:
    fmv.x.w t0, ft0
    ...
    beq  t0, t1, context_b_leaked_a_state

```

Security result

- Context A writes A_SECRET_WORD into ft0.
- The scheduler immediately observes FS=Dirty but stale SD=0.
- The scheduler skips saving or clearing FP state.
- Context B reads ft0 and observes A_SECRET_WORD.
- The PoC exits with code 31.

CVA6 evidence

The evidence log is poc/poc.cva6-h.log:

```

poc.elf *** FAILED *** (tohost = 31) after 2935 cycles
*** [rvfi_tracer] INFO: Simulation terminated after      2923 cycles!

```

tohost = 31 is the PoC's context_b_leaked_a_state path. It is reached only after the scheduler skips the FP save/clear decision and context B reads context A's marker from ft0.

Expected result on a compliant core

The immediate scheduler read after the FP write should report SD=1. The PoC then exits with tohost = 0, representing the path where the context manager recognizes dirty FP state before exposing the register file to another context.

Observed result on CVA6

CVA6 exits with tohost = 31, meaning context B observed context A's FP marker after the scheduler made its decision from stale sstatus.SD=0.

Impact

- Primary: Confidentiality. FP register contents from one context can remain visible to the next context after a stale dirty-summary decision.
- Secondary: Integrity. Skipping the save path can also allow a later context to overwrite FP state that should have been preserved for the previous context.

The impact applies to software that relies on the V-visible SD summary bit for immediate lazy FP context management.

Affected version status

Confirmed affected in a local run of CVA6 revision v5.3.0-168-gcd4c8b17, confirmed at commit cd4c8b17c102, using the cv64a6_imafdch_sv39 Hypervisor configuration.

Fixed: none identified yet.

Fix direction

Compute the `vsstatus.SD` summary from next-state fields after dirty-state updates:

```
if (CVA6Cfg.RVH) begin
  vsstatus_d.sd = (vsstatus_d.xs == riscv::Dirty) | (vsstatus_d.fs == riscv::Dirty);
end
```

The nearby `mstatus.SD` calculation should be reviewed for the same q-state pattern. Regression coverage should include an immediate V-visible `sstatus` read after a VS-mode FP write, followed by a lazy-save style context-switch test that fails if a second context can observe the first context's FP register value.